Some previous years' solved questions

Q) What are keywords and identifiers?

Ans) Keywords- given in note 1.

Identifiers: An identifier is a name that is assigned by the user for a program element such as variable, constants, functions, structures and user-defined data.

There are certain rules for an identifier. These are:-

1. An Identifier can only have alphanumeric characters (a-z , A-Z , 0-9) and underscore(_).
2. The first character of an identifier can only contain alphabet (a-z , A-Z) or underscore (_).
3. Identifiers are case sensitive in C.
4. Keywords are not allowed to be used as Identifiers.
5. No special characters, such as semicolon, period, whitespaces, slash or comma are permitted to be used in or as Identifier.

Q) What is string.h?

Ans) string.h is a header file which contains the prototypes of all standard string library functions such as strlen( ), strcmp( ), strrev( ), strcpy( ), strcat( ), etc. So whenever we need to use any such function in a C program, we have to include string.h header file in our program.

Advantages of pointers

- Pointers provide direct access to memory
- Pointers provide a way to return more than one value to the functions
- Provides an alternate way to access array elements
- Pointers can be used to pass information back and forth between the calling function and called function.
- Pointers allows us to perform dynamic memory allocation and deallocation.
- Pointers allows us to resize the dynamically allocated memory block.

Drawbacks of pointers:

- Uninitialized pointers might cause segmentation fault.
- Dynamically allocated block needs to be freed explicitly. Otherwise, it would lead to memory leak.
- If pointers are updated with incorrect values, it might lead to memory corruption.

Tanumoy Nag

Q) In which header file is NULL pointer defined? How will you include that header file?

Ans) It is defined in the header files- stdio.h, stddef.h, stdlib.h, alloc.h.

Q) What is header file?

Ans) A header file in C programming language is a file with .h extension which contains a set of common function declarations and macro definitions which can be shared across multiple program files. C language provides a set of inbuilt header files which contain commonly used utility functions and macros.

A header file is included in a C program by using the pre-processor directive #include statement. Each header file contains information or declarations for a particular group of functions. Like **stdio.h** header file contains declarations of standard input and output functions available in C. The header file **math.h** contains declarations of mathematical functions available in C. Similarly, the header file **string.h** contains string handling functions.

Q) Is printf a function? If yes, then what does it return?

Ans) printf( ) is a library function. The printf( ) function returns the number of characters printed.

Example 1:

```
int a=500;
printf("%d",printf("\n%d",a));
```

Output: 500 4

This is because the number of characters output is 4. (3 characters for 500 and 1 for \n character.

Example 2:

```
int a=printf("00");

printf("%d",a);
```

Output: 002

Q) Explain with examples the meaning of explicit & implicit type casting.

Ans) A type cast means a conversion from one data type to another.

There are two types of type conversion:

Implicit Type Conversion

This is also known as ‘automatic type conversion’. When the type conversion is performed automatically by the compiler without programmer’s intervention, such type of conversion is known as **implicit type conversion**. It is also known as coercion.

- This type of conversion is done by the compiler on its own, without any external trigger from the user.
- Generally takes place when in an expression more than one data type is present. In such condition type conversion (type promotion) takes place.
- All the data types of the variables are upgraded to the data type of the variable with largest data type.

Example:

```
void main( )
{
int a=50;
float b=a+5.5; //here, integer variable a is implicitly converted to float.
printf("a=%d, b=%f",a,b);
}
```

Output:

a=50, b=55.500000

Explicit Type Conversion

The type conversion performed by the programmer by posing the data type of the expression of specific type is known as explicit type conversion. It is also known as typecasting.

In C, type casting is done in the following form:

(data_type) expression;

where, *data_type* is any valid C data type to which the final result is to be converted, and *expression* may be constant, variable or expression.

Example:

```
void main()
{
float a=5.6;
int b=(int)a+10; //here, float variable a is explicitly converted to integer
printf("b=%d",b);
}
```

Output:

b=15

What is the difference between const char *ptr, char const *ptr and char *const ptr?

const char *ptr : This is a pointer to a constant character.

We cannot change the value pointed by ptr, but we can change the pointer itself.

Example program

```
#include<stdio.h>
int main( )

{
    char a ='A', b ='B';

    const char *ptr = &a;

   //*ptr = b; illegal statement (assignment of read-only location *ptr)

     // ptr can be changed

    printf( "value pointed to by ptr: %c\n", *ptr);

    ptr = &b;

    printf( "value pointed to by ptr: %c\n", *ptr);
}
```

There is no difference between **const char *p and char const *p** as both are pointer to a const char.

char *const ptr : This is a constant pointer to non-constant character. We cannot change the pointer p, but can change the value pointed by ptr.

Example program

```
#include<stdio.h>
int main( )

{
    char a ='A', b ='B';

    char *const ptr = &a;

    printf( "Value pointed to by ptr: %c\n", *ptr);

    printf( "Address ptr is pointing to: %u\n", ptr);
```

```
  //ptr = &b; illegal statement (assignment of read-only variable ptr)

 // changing the value at the address ptr is pointing to

   *ptr = b;

   printf( "Value pointed to by ptr: %c\n", *ptr);


   printf( "Address ptr is pointing to: %u\n", ptr);

}
```

What are the differences between void main( ) and int main( ) ?

Program execution starts from main( ) function. So the operating system calls this function. When some value is returned from main( ), it is returned to operating system.

The void main( ) indicates that the main( ) function will not return any value, but the int main() indicates that the main( ) can return integer type data.

int main( ) returns an integer to the operating system. It indicates the success or failure of program execution. void main( ) does not return a value, does not indicate program success or failure directly. int main( ) is highly portable across different platforms and compilers.

void main( ) may not be portable and accepted by some compilers with warnings or errors.

Q. What are the advantages & disadvantages of recursion?

Ans) <u>Advantages</u>:

1) It allows programmers to take advantage of the repetitive structure present in many problems.
2) Using recursion, the size of the program code can be reduced. Recursion can be a good substitute for iteration where the iterative solution is very complex
3) Complex case analysis and nested loops can be avoided.
4) Recursion can lead to more readable and efficient algorithm descriptions.

<u>Disadvantages</u>:

1) It is slower due to the overhead of maintaining the stack.
2) It usually uses more memory for the stack. It requires extra storage space. The recursive calls and automatic variables are stored on the stack. For every recursive calls separate memory is allocated to automatic variables with the same name.
3) Tracing the code is more difficult in the case of large programs.
4) If recursion is too deep, then there is a danger of running out of space on the stack and ultimately program crashes.

Q. Differences between recursion & iteration.

Ans) The differences between recursion & iteration are as follows:-

1) In recursive approach, the function calls itself again and again until the base condition is met, whereas, in iterative approach, the statements are repeated until the condition fails. An iterative algorithm uses looping statements such as for loop, while loop or do-while loop to repeat the same steps.
2) Recursion terminates when the base condition is met, whereas iteration terminates when the loop-condition fails.
3) Recursion is slower whereas iteration is faster. This is because recursion has the overhead of repeated function calls and the overhead of maintaining the stack. Iteration does not use stack so it is faster than recursion.
4) Recursion uses more memory than iteration. The recursive calls and automatic variables are stored on the stack. For every recursive calls separate memory is allocated to automatic variables with the same name.
5) Recursion reduces the size of the code whereas iteration makes the code longer.

Q. Difference between algorithm & pseudo-code.

Ans) An algorithm is a well defined sequence of steps that provides a solution for a given problem, while a pseudo-code is one of the methods that can be used to represent an

Tanmoy Nag

algorithm. An algorithm has no fixed representation. It can be described in a high-level description, pseudo-code or code in any language.

There are lots of formats used for writing pseudo-codes and most of them borrow some of the structures from popular programming languages such as C, Lisp, FORTRAN, etc. While algorithms can be written in natural language, pseudo-code is written in a format that closely resembles high level programming language structures.

Also, transforming an algorithm presented in pseudo-code to programming code is much easier than converting an algorithm written in natural language.

Example:

Algorithm for Selection Sort

- First find the smallest element in the array.
- Exchange that smallest element with the element at the first position.
- Then find the second smallest element and exchange that element with the element at the second position.
- This process continues until the complete array is sorted.

The psuedocode for above algorithm can be written as:

```
SELECTION-SORT(A)
for j ← 1 to n-1
    smallest ← j
    for i ← j + 1 to n
        if A[ i ] < A[ smallest ]
            smallest ← i
    Exchange A[ j ] ↔ A[ smallest ]
```

Tanumoy Nag

Q. What are high-level language & low-level language?

Ans) A **high-level language** is a programming language such as C, FORTRAN, or Pascal that enables a programmer to write programs that are more or less independent of a particular type of computer. They enable development of a program in a much more user-friendly programming context and is generally independent of the computer's hardware architecture.

Such languages are considered high-level because they are closer to human languages and further from machine languages.

A **low-level language** is a programming language that provides little or no abstraction from a computer's instruction set architecture. Commands in the language map closely to processor instructions. Examples are machine code or assembly language. The word "low" refers to the small or nonexistent amount of abstraction between the language and machine language.

Q. What do you mean by debugging?

Ans) Debugging is the process of locating and resolving of defects or errors within a computer program that prevent correct operation of the program. Debugging checks, detects and corrects errors or bugs to allow proper program operation according to specifications.

Q. Why is nested if-else used?

Ans) The if-else statement allows a choice to be made between two possible alternatives. Sometimes a choice must be made between more than two possibilities. In such cases nested if-else is used. So nested if-else is used when user want to check more than one conditions at a time.

Then give example of the code to find greatest of 3 numbers using nested if-else.

Q. What is an infinite loop? Give example.

Ans) An infinite loop is a piece of coding that lacks a functional exit so that it repeats indefinitely. It is a sequence of instructions in a program which loops endlessly, either due to the loop having no terminating condition, or having one that can never be met, or one that causes the loop to start over. Usually, an infinite loop results from a programming error - for example, where the conditions for exit are incorrectly written.

An example of infinite loop is as follows:

```
int i=1;

while(1)

{

   printf("%d", i);

}
```

Intentional uses for infinite loops include programs that are supposed to run continuously, such as in programming for embedded systems.

Tanumoy Nag

Q. Explain entry-controlled loops & exit-controlled loops.

Ans) **<u>Entry-controlled loop</u>** : Loop where test condition is checked before entering the loop body is known as Entry Controlled Loop. If the test condition is false the very first time, loop body will not be executed**. while loop** and **for loop** are entry-controlled loops because in case of both these loops at first the test condition is checked and if the condition is true, only then the statements within the loop are executed.

[Next explain while loop & for loop from earlier note. Also give example.]

<u>Exit-controlled loop</u>: Loop where test condition is checked after executing the loop body is known as Exit Controlled Loop. Even if the test condition is false the very first time, loop body will be executed once. **do-while loop** is exit-controlled loop because in case of this loop at first the statements within the loop are executed first & after that the test condition is checked.

[Next explain do-while loop from earlier note. Also give example.]

Q. What is an expression in programming?

Ans) An expression is a sequence of one or more operands and zero or more operators that can be evaluated to a single value. Every expression consists of at least one operand and can have one or more operators.

Examples of expressions are:

a+b, x+5, x=y, p=a+b*c, etc.

Tanumoy Nag

Q) WACP to print the summation of the following series:

$1 + 1/2^2 + 1/3^2 + 1/4^2 + \ldots\ldots\ldots\ldots + 1/N^2$

```
#include<stdio.h>
#include<math.h>

void main()
{

    int i,n;
    float sum=0.0;
    printf("\nEnter the number of terms:");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
            sum=sum+1/(float)pow(i,2);
    }
    printf("\nThe sum is %f",sum);
}
```

Q) WACP to reverse an integer number without using strrev() & strlen(), subtract 9 from the reversed number and print the result. [For example, if the given number is 678, then its reverse is 876. Subtracting 9 will give 867.]

```
#include<stdio.h>

void main()
{
    clrscr();
    int n,r,rev=0;
    printf("\nEnter the number:");
    scanf("%d",&n);
    while(n>0)
    {
        r=n%10;
        rev=rev*10+r;
        n=n/10;
    }
    printf("\nThe reversed number is %d",rev);
    rev=rev-9;
    printf("\nThe result is %d",rev);
}
```

Q) What are Unary Operators and Binary Operators?

The operators which operate on a single operand are known as **Unary Operators**. Examples are:

Increment Operator(++) eg i++ (here, the ++ operator works on single operand i)

Decrement Operator(--) eg i - -

Address Of Operator(&) eg &i

The operators which operate on two operands are known as **Binary Operators**. Examples are:

Binary Plus operator(+) eg. a + b (here, + operator works on 2 operands a and b)

Binary Minus operator(-)

Greater than Operator(>)

Q) What is a symbolic constant?

Ans) A symbolic constant is a symbol or name that is used as a substitute for a sequence of character that cannot be changed. The character may represent a numeric constant, a character constant, or a string. When the program is compiled, each occurrence of the symbolic constant is replaced by its corresponding character sequence. Often there are times in a program when we have to use a single value for something over and over. For such purpose, symbolic constants are useful.

Eg. #define RANGE 50 // here RANGE is the

symbolic constant void main()

```
{
        int i;
        for(i=1; i <= RANGE; i++)
                printf("%d",i);
}
```

Q) What is a block?

Ans) Statements which are enclosed within left brace **({)** and the right brace **(})** forms a block.

A block can have any number of statements.

Branching statements, conditional statements, loop control statements such as if, else, switch, for, while forms a block.

Eg.

```
while(n>0)
{
        r = n % 10;
        n = n / 10;
        count ++;
}
```

this is the block

Q) What is a statement terminator?

Ans) A statement terminator is used to demarcate the end of an individual statement. Semi-colon (;) is used as statement terminator in C.

Q) What is a null statement?

Ans) A null statement is a statement containing only a semicolon. No operation is performed when a null statement is executed.

Eg-

```
{
        int i;
        for(i=1;i<=10;i++)
        ;                    // This is a Null statement
}
```

Q) What is a null string?

Ans) A null string has no values. It is an empty character array, one that has not been assigned any elements.

Q) WACP to print integers from 1 to n omitting those integers which are divisible by 10.

```
#include<stdio.h>

void main()
{
    int i,n;
```

Tanumoy Nag

```
    printf("\nEnter the value of n:");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
            if(i%10!=0)
               printf("%d ",i);
    }
}
```

Q) WACP to find the square of the sum of digits of any given positive integer.

```
#include<stdio.h>
#include<math.h>

void main()
{
    int n,r,sum=0;
    printf("\nEnter the number:");
    scanf("%d",&n);

    while(n>0)
    {
          r=n%10;
          sum=sum+pow(r,2);
          n=n/10;
    }
    printf("\nThe sum of square of digits of the given number is %d",sum);
}
```

Determine how much money is in a piggy bank that contains several 50, 25, 20, 10 and 5 paise coins. Use the following values to test the program : five 50 paise, three 25 paise, two 20 paise, one 10 paise and fifteen 5 paise.

```
#include<stdio.h>
void main()
{
```

```
    int sum=0,a,b,c,d,e;
    float r;
    printf("\nEnter the no of 50 paise coins:");
    scanf("%d",&a);
    printf("\nEnter the no of 25 paise coins:");
    scanf("%d",&b);
    printf("\nEnter the no of 20 paise coins:");
    scanf("%d",&c);
    printf("\nEnter the no of 10 paise coins:");
    scanf("%d",&d);
    printf("\nEnter the no of 5 paise coins:");
    scanf("%d",&e);
    sum=(a*50)+(b*25)+(c*20)+(d*10)+(e*5);
    r=(float)sum/100;
    printf("\nTotal money = Rs. %f", r); }
```

Q) WACP to find the biggest of 3 numbers using one extra variable.

```
#include<stdio.h>

void main()
{
    int a,b,c,max;   //max is the extra variable
    printf("\nEnter the 3 no.s");
    scanf("%d%d%d",&a,&b,&c);
    max=a;
    if(b>max)
        max=b;
    if(c>max)
        max=c;
    printf("\nThe maximum number among the 3 given numbers is %d", max);
}
```

Q) WACP to find the LCM of 2 numbers.

```
#include<stdio.h>

void main()
{
   int n1, n2, max;
   printf("\nEnter the two numbers \n");
   scanf("%d %d", &n1, &n2);
   max = (n1 > n2)? n1 : n2;
   while(1)
 {
        if(max % n1 == 0 && max % n2 == 0)
        {
                printf("LCM of %d and %d = %d", n1, n2, max);
                break;
        }
        max++;

 }

}
```

Tanumoy Nag
Tanumoy Nag

Q) WACP to print the sum of 3 digits prime numbers.

```
#include<stdio.h>

void main()
{

    int i,j,sum=0;
    for(i=100;i<=999;i++)
    {
            for(j=2;j<=(i-1);j++)
            {
                    if(i%j==0)
                    break;
            }
            if(j==i)
            {
              sum=sum+i;
            }
    }
    printf("\nThe sum of 3 digits prime numbers is %d",sum);
}
```

C Program to display the following pattern

```
    *
   **
  ***
 ****
*****
```

```
#include<stdio.h>

void main()

{

    int i,j,n,k;

    printf("\nEnter the number of rows:");

    scanf("%d",&n);

    for(i=n;i>=1;i--)

    {

            for(j=1;j<=i-1;j++)

            {
```

```
            printf(" ");
        }
        for(k=1;k<=n-i+1;k++)
        {
            printf("*");
        }
        printf("\n");
    }
}
```

C Program to display the following pattern

```
    *
   ***
  *****
 *******
```

```
#include<stdio.h>
void main()
{
    int i,j,n,k;
    printf("\nEnter the number of rows:");
    scanf("%d",&n);
    for(i=n;i>=1;i--)
    {
        for(j=1;j<=i-1;j++)
        {
            printf(" ");
        }
        for(k=1;k<=n-i+1;k++)
        {
            printf("*");
```

```
        }
         for(k=1;k<=n-i;k++)
         {
              printf("*");
        }
        printf("\n");
}
}
```

WACP to display the following series:

1 + (1 + 2) + (1 + 2 + 3) + (1 + 2 + 3 + 4) + ……

```
#include<stdio.h>

void main()
{
   int n,i,j, sum=0;
   printf("Enter the value of n ");
   scanf("%d",&n);

   for(i=1;i<=n;i++)
   {

          for(j=1;j<=i;j++)
     {

                    sum=sum+j;

          }

       }

       printf("The sum of the series is %d",sum);

}
```